PROGRAMMING IN HASKELL

Topic 13 – Parsers in Haskell

What is a parser

A parser is a function that takes a String and returns one of two things:

or

`Left Error`

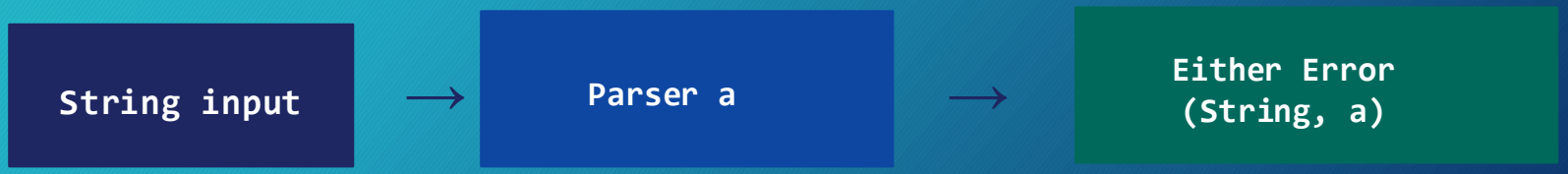
Parsing failed — an error message is returned instead of a value.

`Right (String, a)`

Success — remaining input string plus the parsed value.

Why return the remaining string?

Parsers can be chained: each picks up from where the previous one stopped. Running charP 'a' on "abc" gives back "bc", which a next parser can consume.



Type v newtype

type (alias)

- Just a rename — no new type created
- Fully interchangeable with original
- Cannot define typeclass instances
- No runtime difference at all
- Useful only for readability
- Example: type Error = String

VS

newtype (wrapper)

- Creates a genuinely distinct type
- Compiler treats them as separate types
- Can implement Functor, Monad, etc.
- Zero runtime cost (erased at compile)
- Enables safe abstraction & wrapping
- Example: newtype Parser a = ...

Our three types

1. Error – what we return when parsing fails

```
type Error = String
```

2. Result a – the outcome of any parser

```
type Result a  = Either Error (String, a)
```

3. Parser a – wraps a function from String to Result

```
newtype Parser a = Parser
      { runParser :: String -> Result a }
```

Parsing one character


```
charP :: Char -> Parser Char
charP c =
    Parser $ \input ->
      case input of
        (x:xs) | x == c  ->  Right (xs, x)
        _                ->  Left "character not found"
```

Parsing one character


```
charP :: Char -> Parser Char
charP c =
  Parser $ \input ->
    case input of
      (x:xs) | x == c  ->  Right (xs, x)
      _                ->  Left "character not found"
```

`Signature:` **charP :: Char -> Parser Char** — takes a target character c, returns a Parser Char that looks for it.

`Parser $ \input ->:` Wraps a lambda inside the Parser newtype. The lambda receives the full input string when the parser is run.

`(x:xs) pattern:` Deconstructs the input: x is the first character (head), xs is the remainder (tail). Fails if input is empty.

`| x == c  guard :` Only succeeds if x equals the target c. If not, Haskell falls through to the catch-all clause.

`Right (xs, x) :` Success — returns the remaining string xs and the matched character x, wrapped in Right.

`_  ->  Left ...:` Catch-all — if x ≠ c (or input is empty) parsing fails: Left "character not found".

Using runParser — Worked Examples

a. Find 'a' in "abc" — Success

```
runParser (charP 'a') "abc"
```

```
-> Right ("bc", 'a')
```

First char of "abc" is 'a'.
'a' == 'a' ✓ guard passes.

Returns Right ("bc", 'a'):
remaining = "bc", value = 'a'.

b. Find 'x' in "abc" — Failure

```
"runParser (charP 'x') "abc
```

```
-> Left "character not found"
```

First char of "abc" is 'a'.
'a' == 'x' ✗ guard fails.

Catch-all _ fires.
Returns Left "character not found".

runParser unwraps the Parser newtype and applies the inner function. The result is always an Either — Left for failure, Right for (remaining, value).